



CS4002

Applied Programming

National University of Computer & Emerging
Sciences
Islamabad, Pakistan

Ms. Umarah Qaseem



1

Content

- ▶ Recap of OOP
- ▶ Instantiation – Object Creation
- ▶ Constructors in detail
- ▶ Constructor overloading
- ▶ To learn how to create objects with *new*
- ▶ Encapsulation – Data Hiding
- ▶ To learn the use of *destructor*
- ▶ This pointer
- ▶ Const Functions
- ▶ Separating Declaration from Implementation
- ▶ To study *constructor initializer list*
- ▶ To study *accessor* and *mutator* methods

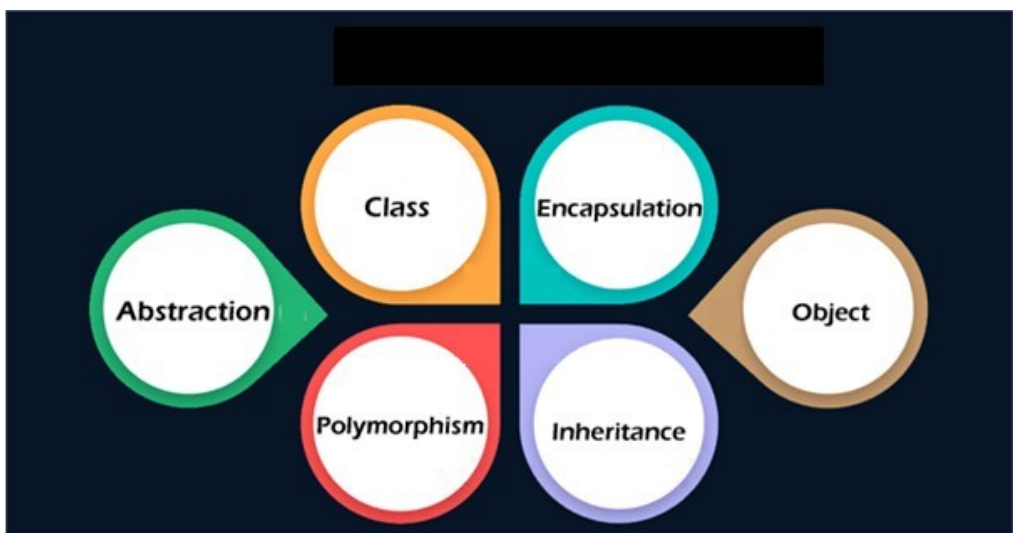
2

Object Oriented Programming – II

3

3

Recap Basic Concepts of **OOP**



4

Basic Concepts of OOP

1. Constructors

5

Object Creation - Instantiation

In C++, you can **assign a name** when **creating an object**.

A **constructor is invoked** when an **object is created**.

The syntax to create an object using the ***no-arg constructor*** is

ClassName **objectName;**

Defining objects in this way means ***creating them***. This is also called ***instantiating them***.

6

A Simple Program – *Object Creation*

```
class Circle
{
    private:
        double radius;

    public:
        Circle()
        { radius = 5.0; }

        double getArea()
        { return radius * radius *
3.14159; }
};
```

```
void main()
{
```

```
    Circle C1;
```

```
    //C1.radius = 10;    can't access private member outside the class
    cout<<"Area of circle = "<<C1.getArea( );
}
```

C1 Object Instance

: C1

radius: 5.0

Allocate memory
for radius

7

Constructors

- A **constructor in a class is a special function used to create an object**. Constructor has **exactly** the **same name** as the **defining class**
- **Constructors** can be **overloaded** (i.e., multiple constructors with **different signatures**) [**Purpose:** *making it easy to construct objects with different initial data values*].
- A **class** may be **declared without constructors**. In this case, a **no-argument constructor with an empty body** is **implicitly** declared in the class known as **default constructor**
- **Note:** Default constructor is provided automatically **only if no constructors are explicitly declared in the class**.

8

Constructors' Properties

- Constructors *must have the same name as the class* itself.
- Constructors *do not have a return type*—*not even void*.
- Constructors *play the role of initializing objects*.

9

Object Member Access Operator

After object creation, its **data** and **functions** can be **accessed** (invoked) using:

the **(.) operator**, also known as the *object member access operator*.

objectName.dataField references a data field in the object

objectName.function() invokes a function on the object

10

A Simple Program – Accessing Members

```
class Circle
```

```
{
```

```
    private:
        double radius;
```

```
    public:
```

```
        Circle()
```

```
        { radius = 5.0; }
```

```
        double getArea()
```

```
        { return radius * radius * 3.14159; }
```

```
};
```

C1 Object Instance

: C1

radius: 5.0

Allocate memory
for radius

```
void main( )
```

```
{
```

```
    Circle C1;
```

```
    //C1.radius = 10; can't access private member outside the class
```

```
    cout<<"Area of circle = "<<C1.getArea();
```

```
}
```

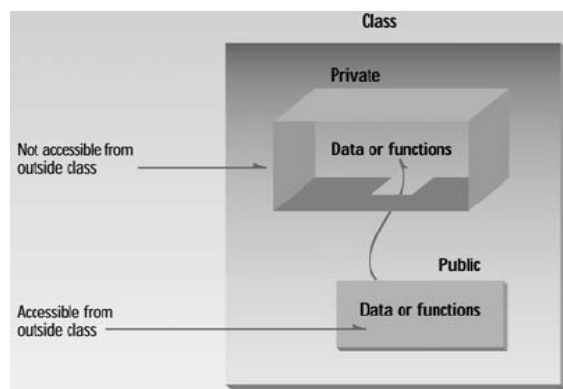
11

Access Modifiers

Access modifiers are used to set access levels for *classes, variables, methods, and constructors*

private, public, and protected

In C++, *default accessibility is private*



12

Data Hiding - Data Field Encapsulation

A key feature of OOP is **data hiding**

data is concealed within a class so that it cannot be accessed mistakenly by functions outside the class.

To prevent direct modification of class attributes (outside the class), the primary mechanism for hiding data is to put it in a class and make it private using *private* keyword. **This is also known as data field encapsulation.**

13

Hidden from Whom?

Data hiding means hiding data from parts of the program that don't need to access it. More specifically, one class's data is hidden from other classes.

Data hiding is designed to protect well-intentioned programmers from mistakes.

14

A Simple Program – Accessing *Member Function*

```
class Circle
```

```
{
```

```
    private:
```

```
        double radius;
```

```
    public:
```

```
    Circle()
```

```
    {    radius = 5.0;    }
```

```
    double getArea()
```

```
    {    return radius * radius *
```

```
    3.14159; }
```

```
};
```

C1 Object Instance

: C1

radius: 5.0

Allocate memory
for radius

```
void main()
```

```
{
```

```
    Circle C1;
```

```
    //C1.radius = 10; can't access private member outside the class
```

```
    cout<<"Area of circle = "<<C1.getArea();
```

```
}
```

15

A Simple Program – *Default Constructor*

```
class Circle
```

```
{
```

```
    private:
```

```
        double radius;
```

```
    public:
```

```
    //Default Constructor
```

```
    Circle()
```

```
    { No Constructor Here }
```

```
    double getArea()
```

```
    {    return radius * radius * 3.14159; }
```

```
};
```

C1 Object Instance

: C1

radius: Any Value

Allocate memory
for radius

```
void main()
```

```
{
```

```
    Circle C1;
```

```
    //C1.radius = 10; can't access private member outside the class
```

```
    cout<<"Area of circle = "<<C1.getArea();
```

```
}
```

16

Object Construction with Arguments

- The syntax to declare an object using a constructor with arguments is:

ClassName objectName(arguments);

- For example, the following declaration creates an object named ***circle1*** by invoking the ***Circle*** class's **constructor** with a specified radius **5.5**.

Circle circle1(5.5);

17

A Simple Program – Constructor with Arguments

```
class Circle
{
    private:
        double radius;
    public:
        Circle(double rad)
        { radius = rad; }
        double getArea()
        { return radius * radius * 3.14159; }
};
```

C1 Object Instance

: C1
radius: 9.0

Allocate memory
for radius

```
void main()
{
    Circle C1(9.0);
    //C1.radius = 10;    can't access private member outside the class
    cout<<"Area of circle = "<<C1.getArea();
}
```

18

Output of the following Program?

```
class Circle
{
    private:
        double radius;
    public:
        Circle(double rad)
        {    radius = rad;    }
        double getArea()
        { return radius * radius * 3.14159; }
};
```

```
void main()
{
    Circle C1;
    cout<<"Area of circle = "<<C1.getArea();
}
```

19

Constructor Overloading

```
class Circle
{
    private:
        double radius;
    public:
        Circle ()
        {    radius = 1;    }
        Circle(double rad)
        {    radius = rad;    }
        double getArea()
        { return radius * radius * 3.14159; }
};
```

```
void main()
{
    Circle C2(8.0);
    Circle C1;
    cout<<"Area of circle = "<<C1.getArea();
}
```

20

The *this* Pointer

- *this* keyword is a special built-in pointer that references to the calling object.
- The *this* pointer is passed as a hidden argument to all non-static member function calls and is available as a local variable within the body of all non-static functions.
- Can be used to access instance variables within constructors and member functions

21

Without *this* Pointer

```
class Circle
{
    private:
        double radius;

    public:
        Circle(double rad)
        { radius = rad; }
        double getArea()
        { return radius * radius * 3.14159; }
};
```

```
void main()
{
    Circle C1(99.0);
    cout<<"Area of circle = "<<C1.getArea();
}
```

23

The *this* Pointer

```
class Circle
{
    private:
        double radius;
    public:
        Circle(double radius)
        {
            this->radius = radius;
        }
        double getArea()
        {
            return radius * radius * 3.14159;
        }
};
```

```
void main()
{
    Circle C1(99.0);
    cout<<"Area of circle = "<<C1.getArea();
}
```

24

Other use of *this* Pointer

```
class Circle
{
    private:
        double radius;
    public:
        Circle(double radius)
        {
            this->radius = radius;
        }
        double getArea()
        {
            return this->radius * this->radius * 3.14159;
        }
};
```

```
void main()
{
    Circle C1(99.0);
    cout<<"Area of circle = "<<C1.getArea();
}
```

25

Member Functions

Separating Declaration from Implementation

C++ allows to separate class declaration from implementation

Class declaration describes the contract of the class and the class implementation implements the contract

The class declaration simply lists all data fields, constructor prototypes, and the functions

The class implementation implements the constructors and functions

26

Member Functions

Separating Declaration from Implementation

```
class Circle
{
    private:
        double radius;

    public:
        Circle(double radius)
        {    this->radius = radius;    }
        double getArea( ); // Not implemented yet
};

double Circle::getArea()
{ return this->radius * radius * 3.14159; }
```

```
void main()
{
    Circle    C1(99.0);
    cout<<"Area of circle = "<<C1.getArea();
}
```

27

Member Functions

Separating Declaration from Implementation

```
#include <iostream>
using namespace std;
```

```
class Student {
    string name;
    int age;
```

```
public:
    //Constructor Declaration (only)
    //Member Function Declaration (only)
};
```

```
//complete code
```

```
int main() {
    Student s1("Ali", 21); // constructor runs
                             here
    s1.display();
    return 0;
}
```

28

Member Functions

Separating Declaration from Implementation

```
#include <iostream>
using namespace std;
```

```
class Student {
    string name;
    int age;
```

```
public:
    //Constructor Declaration (only)
    Student(string n, int a);

    //Member Function Declaration (only)
    void display();
};
```

```
// Constructor Definition (outside the class)
Student::Student(string n, int a) {
    name = n;
    age = a;
    cout << "Constructor called for " << name << endl;
}
```

```
// Member Function Definition (outside the class)
void Student::display() {
    cout << "Name: " << name << ", Age: " << age << endl;
}
```

```
int main() {
    Student s1("Ali", 21); // constructor runs
                             here
    s1.display();
    return 0;
}
```

29

const Member Functions

```
class Example
{
private:
    int var1;
    int var2;
public:
    Example(int var1,int var2)
    {    this->var1 = var1;
        this->var2 = var2;
    }
    void show() const
    { cout<<"var1 = "<<this->var1;
      cout<<"var2 = "<< this->var2;
      // var1 = 90;
      // var2 = 50;
    }
};
```

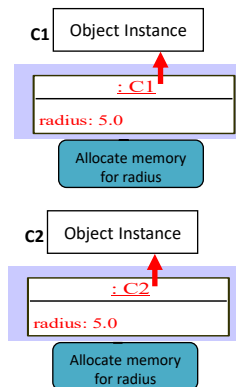
```
void main()
{
    Example ex1(10, 60);
    Example ex2(170, 30);
    ex1.show();
    ex2.show();
}
```

30

A Simple Program – Object Creation using new

```
class Circle
{
private:
    double radius;
public:
    Circle()
    {    radius = 5.0; }
    double getArea()
    { return radius * radius *
      3.14159; }
};
```

```
void main()
{
    Circle c1;
    Circle *c2 = new Circle();
    cout<<"Area of circle = "<<c1.getArea()<<c2->getArea();
}
```



31

Destructors

A **destructor** is a member function with the same name as its class prefixed by a **~ (tilde)**.

Destructors are usually used to de-allocate memory and do other cleanup for a class object and its class members when the object is destroyed.

A destructor takes no arguments and has no return type.

Destructors cannot be declared **const**, **volatile**, **static**. A destructor can be declared **virtual** or **pure virtual**.

32

Destructor

If no user-defined destructor exists for a class and one is needed, the compiler implicitly declares a destructor:

This implicitly declared destructor is a public member of its class.

33

Destructor - Example

```

class Circle
{
    private:
        double radius;
    public:
        Circle();
        Circle(double);
        double getArea();
        ~Circle();
};

```

Class
declaration

```

void main()
{
    Circle c1;
    Circle *c2 = new Circle(10.7);
    cout<<"Area of circle = "<<c1.getArea();
    cout<<"Area of circle = "<<c2->getArea();
}

```

```

Circle::Circle()
{
    radius = 1.0; }
Circle::Circle(double rad)
{
    radius = rad; }
double Circle::getArea()
{
    return radius * radius * 3.14; }
Circle::~Circle() { cout<<"Destructor called"; }

```

Class
Implementation

34

Constructor_INITIALIZER

```

Circle(): radius(1)
{
}

```

(a)

Same as

```

Circle()
{
    radius = 1;
}

```

(b)

35

Member Initialization List

One task of the **Constructor** is to **initialize data members**:

```
Circle( ) { radius = 1.0; }
```

But it is *not preferred approach*. **The preferred approach** is:

```
Circle( ): radius(1.0)
{ }
```

If **multiple members** are to be initialized, separate them by **commas**. The result is the initializer list (also known as **member-initialization list**)

```
Circle( ): radius(1.0), area(0.0)
{ }
```

Member
Initialization List

36

Reasons for Constructor Initializers

In the initializer list, **members initialized with given values before the constructor even starts to execute**

This is important in some situations e.g. the **initializer list is the only way to initialize const member data**

37

Reasons for Constructor Initializers

```
#include<iostream>
using namespace std;

class Test {
    const int t;
public:
    Test(int t):t(t) {} //Initializer list must be used
    int getT() { return t; }
};

int main() {
    Test t1(10);
    cout<<t1.getT();
    return 0;
}

/* OUTPUT:
10
*/
```

38

Accessor (getter) and Mutator (Setter) Functions

A **get** function is referred to as a **getter** (or **accessor**),

A **set** function is referred to as a **setter** (or **mutator**).

A get function has the following header:

```
returnType  getProperty() { }
```

A set function has the following header:

```
void setPropertyName(dataType  propertyValue) { }
```

39

Example

```
class Book{
private:
    int bookID, totalPage;
    float price;
public:
    Book():bookID(0),totalPage(0),price(0.0)
    {
    }
    void setBookID(int bid){ bookID = bid; }
    void setPages (int pn){ totalPage = pn; }
    void setPrice(float p){ price = p; }

    int getBookID(){ return bookID; }
    int getPages(){ return totalPages; }
    float getPrice(){ return price; }
};
```

**Mutator
Functions**
(i.e., **setter**)

**Accessor
Functions**
(i.e., **getters**)